

Задание 3. Проектирование архитектуры микросервисной информационной системы

Москалец Данила Алексеевич

- 1) Архитектурное видение и контекст;
- 2) Логическая архитектура и доменное моделирование;
- 3) Архитектура микросервисов;
- 4) Архитектура данных;
- 5) Архитектура API;
- 6) Качественные атрибуты;
- 7) Инфраструктура и развёртывание.

ВВЕДЕНИЕ

Sangel – это прототип системы для агрегирования групп быстрого реагирования охранных предприятий (ЧОП). Система позволяет клиентам создавать заявки на вызов охраны, группам принимать эти заявки и менять их статус, а операторам контролировать весь процесс.

1 АРХИТЕКТУРНОЕ ВИДЕНИЕ И КОНТЕКСТ

1.1 Описание целей системы

Бизнес-цели:

- Создать заявку и найти свободную группу ЧОП в течение нескольких минут;
- Позволить клиентам отслеживать статус заявки в реальном времени;
- Дать операторам контроль над всеми заявками и группами.

Метрики:

- Время ответа при создании заявки: менее 1 секунды;
- Доступность: не менее 99% времени;
- Поддержка 100-200 одновременных пользователей.

Нефункциональные требования:

- Надёжность: система должна работать даже при сбое одного компонента;
- Масштабируемость: возможность добавить новые ЧОП без переделки ядра;
- Безопасность: разные права доступа для разных ролей.

1.2 Контекстная диаграмма

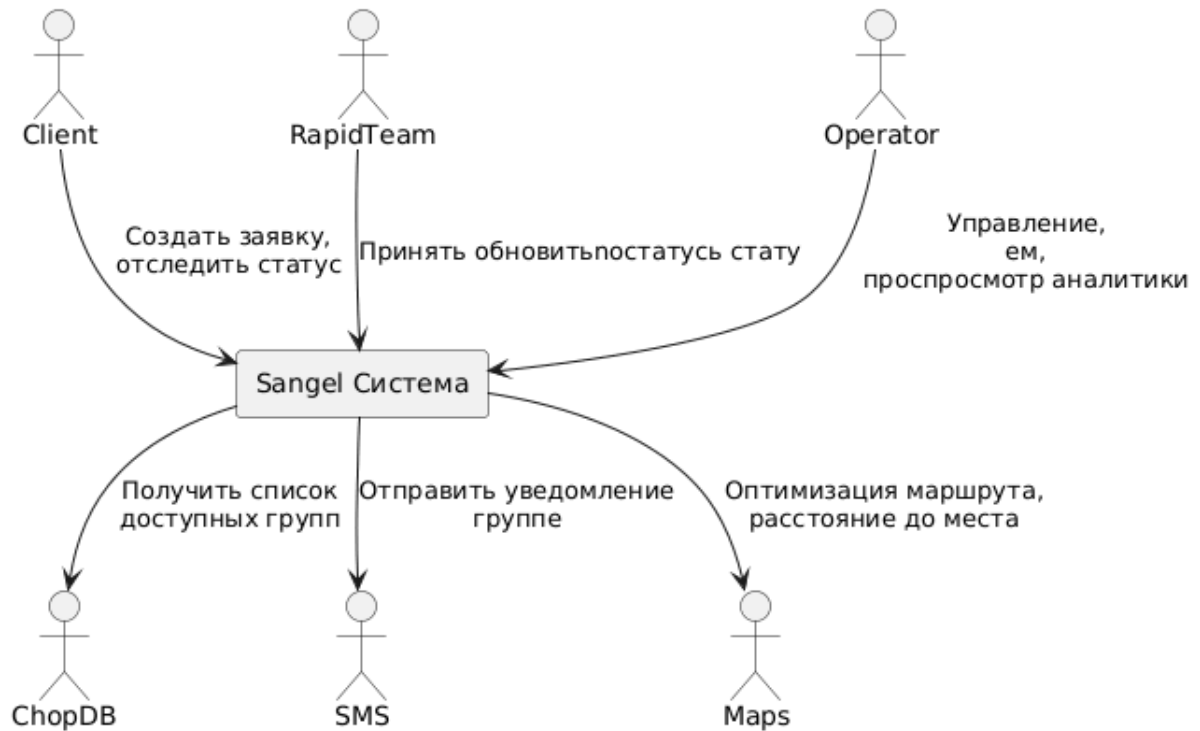


Рисунок 1 – Контекстная диаграмма

Описание: Система связывает трёх типов пользователей: клиента, группу быстрого реагирования и оператора. Главное ограничение: одна группа может работать максимум с одной заявкой одновременно. Система отправляет уведомления через SMS и использует карты для определения расстояния.

Вывод: Контекстная диаграмма показала, что система – это по сути система распределения задач между исполнителями. Главная сложность не в технологиях, а в синхронизации состояния между заявками и доступностью групп. Я понял, что нужно чётко разделить логику: управление заявками отдельно от управления группами.

2 ЛОГИЧЕСКАЯ АРХИТЕКТУРА И ДОМЕННОЕ МОДЕЛИРОВАНИЕ

2.1 Выделение доменных областей

Система состоит из трёх независимых доменов:

1. Order Domain (домен заявок)

- Отвечает за: создание заявок, смену статусов, историю;
- Основные сущности: Ticket (заявка);
- Статусы: NEW → ASSIGNED → IN_PROGRESS → COMPLETED/CANCELLED.

2. Dispatch Domain (домен диспетчеризации)

- Отвечает за: распределение заявок между группами, управление статусом групп;
- Основные сущности: RapidTeam (группа), Assignment (назначение);
- Главное правило: одна активная заявка на группу.

3. Auth Domain (домен аутентификации)

- Отвечает за: управление пользователями, проверка прав, выдачу токенов;
- Основные сущности: User, Role (CLIENT, RAPID_TEAM, OPERATOR).

2.2 Главные сущности

Ticket (заявка):

- ID заявки;
- Описание проблемы;
- Местоположение (координаты);
- Приоритет (LOW, MEDIUM, HIGH);
- Статус (NEW, ASSIGNED, IN_PROGRESS, COMPLETED, CANCELLED);

- ID назначенной группы;
- Время создания.

RapidTeam (группа быстрого реагирования):

- ID группы;
- Название и контакты;
- Текущий статус (IDLE, BUSY);
- Местоположение;
- Активная заявка (если есть).

2.3 Главные бизнес-процессы

Процесс 1: Создание и назначение заявки

- 1) Клиент создаёт заявку через форму;
- 2) Система ищет свободную группу рядом с местом заявки;
- 3) Если группа найдена – система назначает заявку, группа видит уведомление;
- 4) Если групп нет – заявка остаётся в очереди, система пересчитывает каждые 30 секунд.

Процесс 2: Завершение заявки

- 1) Группа завершает работу и отмечает заявку как COMPLETED;
- 2) Группа становится свободной (статус IDLE);
- 3) Система ищет следующую заявку в очереди.

Вывод: Выделение доменов помогло понять, что заявки и группы – это две разные истории. Без явного разделения система быстро превратилась бы в запутанный код. DDD показал, что правильная архитектура – это не только про технологии, но про понимание бизнеса.

3 АРХИТЕКТУРА МИКРОСЕРВИСОВ

3.1 Состав микросервисов

Сервис	Отвечает за	Основные операции	Данные
OrderService	Управление заявками	Создать, обновить статус, отменить, получить	Tickets, History
DispatchService	Распределение заявок	Найти свободную группу, назначить заявку, освободить группу	Teams, Assignments
AuthService	Аутентификация	Логин, проверка токена, получить пользователя	Users, Roles
APIGateway	Маршрутизация	Маршрутизировать запросы, проверить права	

3.2 Диаграмма взаимодействия

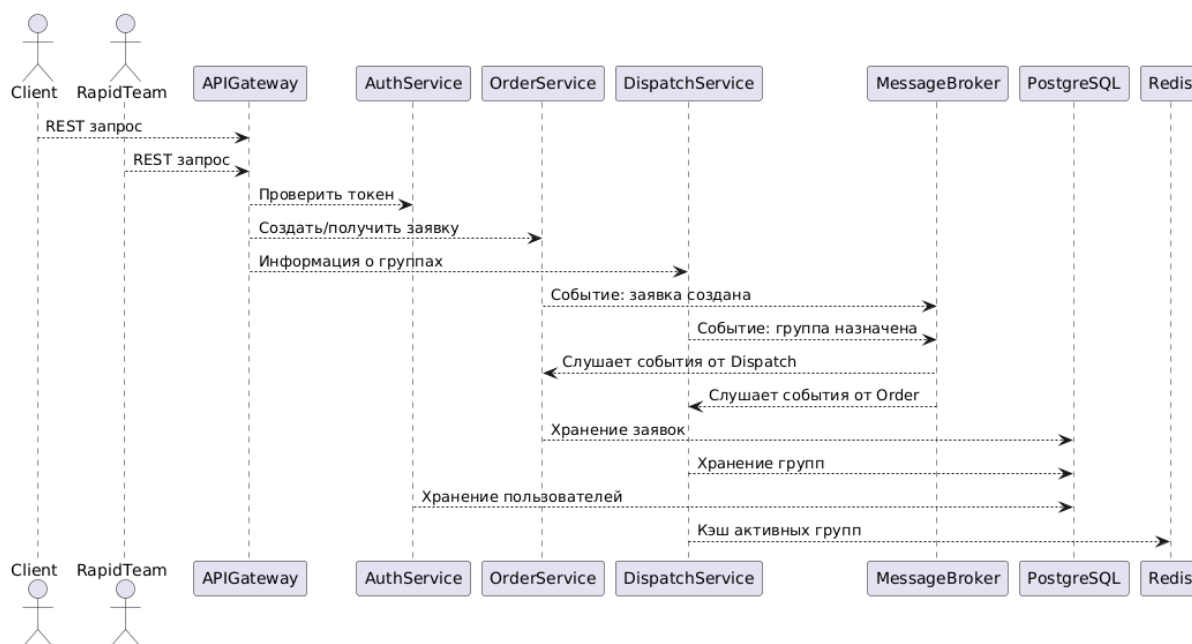


Рисунок 2 – Диаграмма взаимодействия

Связи:

- Синхронные (REST): APIGateway → сервисы, быстро;
- Асинхронные (события): через Message Broker (RabbitMQ), для слабой связанности.

3.3 Почему такое разделение?

О

r
d
e

- Разные операции имеют разную нагрузку (создание заявки vs поиск свободной группы);
- Разные причины для изменения (бизнес-логика заявок vs алгоритм распределения);
- Возможность масштабировать независимо.

AuthService отдельно:

- Может переиспользоваться в других системах ЧОП;
- Проще добавить интеграцию с LDAP или OAuth.

Альтернатива – один микросервис на всё: проще разворачивать, но сложнее развивать и масштабировать.

Вывод: Правильная граница микросервиса – это не размер кода, а наличие отдельной причины для изменения. Я осознал, что разбиение на микросервисы ради микросервисов – это ошибка. Каждый сервис должен иметь чёткую ответственность.

4 АРХИТЕКТУРА ДАННЫХ

4.1 Выбор хранилищ

Сервис	СУБД	Почему
OrderService	PostgreSQL	Нужна консистентность и сложные запросы JOIN
DispatchService	PostgreSQL + Redis	Р о
AuthService	PostgreSQL	Стандартная БД для пользователей

Почему не SQLite как в прототипе?

- S
- PostgreSQL масштабируется до тысяч;
- R

Почему Redis?

- Быстрый поиск доступной группы по геолокации;
- Данные горячие (часто читаются, редко пишутся);
работает до 50-100 одновременных соединений;
- Можно пересоздать из PostgreSQL если сбой.

4.2 Согласованность между сервисами

Проблема: OrderService создал заявку, DispatchService должен её назначить. Как гарантировать, что оба знают о назначении?

Решение – Saga паттерн:
поддерживает репликацию для надёжности.

- 1) O
- 2) D
- 3) D
- 4) D
- 5) D
- 6) Если что-то сломалось – оба отказываются, система остаётся консистентной.

Результат: система достигает консистентности через последовательность событий, а не через одну большую транзакцию.

Б

Б

Б

Вывод: Распределённые транзакции в микросервисной архитектуре – это сложнее, чем в монолите. Но Saga паттерн показывает, что это возможно. Я понял, что нельзя просто перенести логику монолита в микросервисы – нужно думать по-другому.

5 АРХИТЕКТУРА API И ИНТЕГРАЦИЙ

5.1 Внешний API для клиентов

Р

О

Параметры: описание, местоположение, приоритет

С: ID заявки, статус

Т

G

Е

Ответ: заявка с текущим статусом, информация о группе

Т

P

У

Параметры: новый статус (COMPLETED, CANCELLED)

tickets – создать заявку

Г

а

и

а

е

т

р

а

и

а

е

т

р

а

и

а

е

т

р

а

и

а

е

т

р

а

и

а

е

т

р

а

и

а

е

т

р

а

и

а

е

т

р

а

и

а

е

т

р

а

и

а

е

т

р

а

и

а

е

т

р

а

и

а

Полное описание всех эндпоинтов находится в *openapi.yml*.

Основные группы эндпоинтов:

teams – список доступных групп (для оператора)

– *POST /auth/register* – регистрация;

– *P*

для внешних клиентов

е – получить статус заявки

tickets (управление заявками):

T

– *POST /tickets* – создать заявку;

status – обновить статус (для группы)

– *G*

и

с – *GET /tickets/{id}* – получить заявку;

а

– *PUT /tickets/{id}/status* – обновить статус;

т

– *POST /tickets/{id}/cancel* – отменить заявку (отменяется по ролям);

profile – вход и профиль пользователя.

и

– *GET /tickets/{id}/history* – история изменений.

о

n

Teams (управление группами ЧОП):

вход и профиль):

- *GET /teams* – список групп;
- *G*
- *E*
- *P*
- *GET /teams/{id}/active-ticket* – активная заявка группы.

ChOP Management (администрирование):

- *GET /chops* – список ЧОП (только OPERATOR);

– *P*

Statistics (аналитика):

- *GET /statistics/tickets* – статистика по заявкам;
- *GET /statistics/teams* – статистика по группам;
- *chops* – добавить новое ЧОП (только OPERATOR).
- *status* – обновить статус группы;

Контроль доступа на уровне API:

- *location* – обновить местоположение;
- Каждый эндпоинт проверяет роль пользователя (CLIENT, OPERATOR, DISPATCHER);
- C
- OPERATOR видит всё;
- Все запросы требуют JWT токена (кроме регистрации и логина).

5.3 Взаимодействие между микросервисами

OrderService → DispatchService (REST):

- Запрос: «Найди и назначь свободную группу для заявки X»;
- Ответ: ID назначенной группы или ошибка «нет свободных групп»;
- Таймаут: 5 секунд (если DispatchService не ответил – заявка остаётся в очереди).

Через Message Broker (события):

- OrderService публикует: «TicketCreated»;
- DispatchService публикует: «TeamAssigned»;

слушает: «TeamAssigned» для обновления истории.

5.4 Интеграция с внешними системами

S

M

Карты (Яндекс): система запрашивает расстояние от группы до места заявки

уведомления: система отправляет SMS группе через внешний API

A

n

t

Вывод: API должен быть гибким, но строгим – должны быть понятные правила что может делать каждая роль. При проектировании API я осознал, что разделение прав на уровне эндпоинтов важнее чем на уровне БД, потому что клиент часто не предполагает, что у него нет доступа. Anti-Corruption Layer помогает избежать того, чтобы система засорялась code'ом для интеграции с

f

u

p

s

t

a

i

c

o

y

n

системами.

L

a

y

e

r

если интегрируемся с legacy системой ЧОП, оборачиваем её API в адаптер, чтобы не засорять наш код

6 КАЧЕСТВЕННЫЕ АТТРИБУТЫ

6.1 Масштабируемость

Где узкие места?

- О
- Решение: несколько реплик сервиса, их нагрузка балансируется;
- D
- i
- Решение: кэш в Redis с индексом по геолокации;
- Р
- Решение: отправка SMS может быть медленной;
- Решение: асинхронно, очередь в RabbitMQ.

6.2 Надёжность

Паттерны для устойчивости к сбоям:

- h
- T
- i
- B
- E
- F
- e

6.3 Безопасность

При ошибке повторяем запрос несколько раз:
Поиск свободной группы может быть медленным (нужно проверить
Аутентификация: пользователь логинится, получает токен, отправляет
1000 групп). Если сервис не ответил за 5 секунд, считаем это ошибкой, не ждём
его в каждом запросе
дальше;

Авторизация (разные права для разных ролей):

- k
- E
- Если главный путь не работает, используем запасной (например,
- отправляем заявку в очередь вместо прямого вызова).

Защита: все запросы только через HTTPS, API ключи хранятся на
сервере

Если сервис часто падает, останавливаем попытки к нему на время;
может создавать заявки только себе, видит только свои заявки;

E

6.4 Наблюдаемость

Логи: структурированные логи со временем, названием сервиса, типом события (для анализа)

Метрики: сколько заявок создано, среднее время обработки, количество ошибок

Трассировка: когда запрос идёт через несколько сервисов, можем увидеть, где он залипал

Алерты: если что-то ломается (много ошибок, медленный ответ), система уведомляет администратора

Вывод: Наблюдаемость в микросервисной архитектуре критична. Когда система состоит из 10 сервисов, логи в одном файле уже не помогают. Правильные метрики – это глаза администратора.

7 ИНФРАСТРУКТУРА И РАЗВЁРТЫВАНИЕ

7.1 Выбор платформы

Облако (AWS, Yandex Cloud) + Docker контейнеры + Kubernetes

Почему не на своих серверах?

- Нужно администрировать железо (дорого, требует специалистов);
- Сложно масштабировать (нужно покупать новый сервер).

Почему Kubernetes?

- Автоматически запускает несколько копий каждого микросервиса;
- Автоматически перезапускает упавший сервис;
- Распределяет нагрузку между копиями.

Альтернатива: просто виртуальные машины (проще начать, но сложнее масштабировать)

7.2 Окружения

D

T

S

локальное): всё на компьютере разработчика через Docker Compose

P

автоматические тесты на каждый PR

7.3 Выпуск новых версий

Стратегия: Canary deployment.

промежуточное): копия продакшена, здесь тестируем перед выпуском
боевое): реальная система для пользователей, высокая доступность (несколько копий каждого сервиса)

1) Новая версия запускается для 5% пользователей;

2) Мониторим ошибки: если нормально → для 25% → 50% → 100%;

3) Если ошибок много → откатываемся на старую версию.

Преимущество: новая версия сломалась? Затронула только 5% пользователей, остальные не заметили.

7.4 Непрерывная интеграция (CI/CD)

Процесс автоматический:

- 1) Разработчик пушит код в GitHub;
- 2) Автоматически запускаются тесты;
- 3) Если тесты прошли → собирается Docker образ;
- 4) Образ отправляется в registry;
- 5) Автоматически развёртывается в stage окружение;
- 6) Если всё хорошо → можно вручную нажать кнопку Deploy to Prod;
- 7) На prod запускается canary deployment.

Вывод: Инфраструктура и автоматизация – это то, что превращает архитектуру из диаграмм в работающую систему. Правильный выбор платформы позволяет сосредоточиться на разработке, а не на администрировании серверов. CI/CD pipeline гарантирует, что в production попадает только тестированный код.

ЗАКЛЮЧЕНИЕ

Микросервисная архитектура Sangel решает реальные проблемы растущей системы:

- 1) **Независимое развёртывание** – команда разработчиков может выпускать изменения в OrderService без влияния на DispatchService;
- 2) **Масштабируемость** – если нагрузка на поиск групп выросла, мы масштабируем только DispatchService, а не всю систему;
- 3) **Надёжность** – если DispatchService упал, новые заявки создаются, они просто останутся в очереди (при восстановлении сервиса будут обработаны);
- 4) **Гибкость** – можно заменить DispatchService на другую реализацию без изменения OrderService.

Но эта архитектура сложнее монолита:

- Нужно тщательнее мониторить систему;
- Сложнее отлаживать проблемы (запрос проходит через несколько сервисов);
- Нужно думать о согласованности данных иначе.

Главное, что я извлёк: архитектура – это выбор компромиссов. Монолит проще разворачивать, микросервисы проще масштабировать. Для каждого проекта нужно выбрать то, что подходит именно ему.

ПРИЛОЖЕНИЯ

Приложение Диаграмма состояний заявки

Приложение Пример взаимодействия сервисов (сценарий создания заявки)

Диаграмма состояний заявки



Рисунок А.1 – Диаграмма взаимодействия

Пример взаимодействия сервисов (сценарий создания заявки)

1. Клиент: POST /api/tickets
└ OrderService: создаёт ticket, публикует событие "TicketCreated"
2. DispatchService слушает событие
└ DispatchService: ищет свободную группу, резервирует её
3. DispatchService вызывает OrderService
└ OrderService: обновляет ticket.status = "ASSIGNED"
4. DispatchService публикует "TeamAssigned"
└ Клиент получает обновление через WebSocket
5. Группа видит уведомление, принимает заявку
└ OrderService: обновляет ticket.status = "IN_PROGRESS"